Product Vision

Our Star Wars Themed Attraction Reviews website is a Yelp-style web application for Star Wars-themed attractions at Disney parks worldwide. Users can discover attractions, view them on an interactive globe, rate them 1–5 stars, and write detailed reviews, all within a themed, immersive interface.

Technology Stack

Backend

- **Runtime & Framework:** ASP.NET Core 10 (C#) Web API
- **ORM:** Entity Framework Core 10 with Npgsql provider
- **API Documentation:** Swagger UI via Swashbuckle / OpenAPI
- **Testing:** xUnit with coverlet for code coverage

Frontend

- **UI Framework:** Bootstrap 5 (learned in Sprint 2)
- **Design:** Figma (exported to HTML/CSS in Sprint 2)
- **Map:** Google Maps / Google Earth JavaScript API for interactive globe with attraction pins

Infrastructure

- **Locations Database:** PostgreSQL — stores parks, attractions, categories, and images
- **User/Site Database:** PostgreSQL (separate instance) — stores user accounts and reviews
- **Containerization:** Docker + Docker Compose for local development and deployment

- **CI/CD:** GitHub Actions — automated build, test, and deploy pipeline (Sprint 2)

Core Features (MVP)

Interactive Map

The home page displays a Google Earth-style globe with clickable pins for every park in scope. Clicking a pin zooms into that park and navigates to its detail page.

Attraction Detail Pages

- Name, park, location (city & country), and category
- Description and photo gallery (image URLs stored as JSONB)
- Cached average star rating and review count (updated via database trigger)
- Tags (e.g. "outdoor", "family-friendly") stored as JSONB array

Reviews & Ratings

- 1–5 star ratings per attraction
- Written review body (minimum 20 characters, enforced in application layer)
- Optional visit date
- One review per user per attraction (enforced via unique database index)

User Accounts

- Register with username, email, and bcrypt-hashed password
- Login / logout
- View and manage own reviews

Browse & Search

- Filter by park, category, and rating

- Full-text search across attraction names and review bodies

Attraction Categories

- **Rides & Experiences —** e.g. Millennium Falcon: Smugglers Run, Star Wars: Rise of the Resistance
- **Restaurants & Cafes —** e.g. Docking Bay 7 Food and Cargo, Oga's Cantina, Ronto Roasters
- **Shops & Merchandise —** e.g. Dok-Ondar's Den of Antiquities, Savi's Workshop, Droid Depot
- **Shows & Entertainment —** e.g. character meet-and-greets, live DJ performances at Oga's Cantina

Parks in Scope

- **Star Wars: Galaxy's Edge** — Disneyland, Anaheim CA (park_gge_dla)
- **Star Wars: Galaxy's Edge** — Walt Disney World, Orlando FL (park_gge_wdw)
- **Star Wars: Galaxy's Edge** — Disneyland Paris (park_gge_dlp)
- **Star Wars Launch Bay** — Hollywood Studios, Walt Disney World (park_slb_wdw)

Database Schema

Two separate PostgreSQL databases are used:

- **Locations DB** — parks, attractions, categories, attraction_categories
- **User/Site DB** — users, reviews (future sprint)

Locations Database Tables

- **parks** — id (varchar PK), name, resort, city, country, lat/lng coordinates
- **categories** — id (UUID PK), name (unique), description

- **attractions** — id (UUID PK), park_id (FK), name, description, lat/lng, image_urls (JSONB), tags (JSONB), avg_rating, review_count, created_at
- **attraction_categories** — junction table linking attractions to categories

User/Site Database Tables (Future Sprint)

- **users** — id (UUID PK), username (unique), email (unique), password (bcrypt), created_at
- **reviews** — id (UUID PK), attraction_id (FK), user_id (FK), rating (1–5), body (min 20 chars), visit_date, created_at, updated_at

API Design

The REST API is documented via Swagger UI at /swagger. All endpoints return JSON. The Locations API serves data from the Locations DB only.

Locations API Endpoints

- `GET /parks` — list all parks
- `GET /parks/{id}` — park detail
- `GET /attractions` — list attractions (supports ?parkId, ?category, ?search filters)
- `GET /attractions/{id}` — attraction detail
- `GET /categories` — list all categories
- `GET /attraction/image/{name}` — serve attraction image binary

Sprint 2 Goals

- **STAR-15** Update Spec Document (this document)
- **STAR-16** Learning Bootstrap — team ramp-up on Bootstrap 5 for UI components
- **STAR-17** Stub out API — scaffold controller and route structure

- **STAR-18** Basic Homepage Built — globe map with park pins
- **STAR-19** Building Tabs — tabbed navigation for park/category browsing
- **STAR-20** CI/CD — GitHub Actions pipeline for automated build and test
- **STAR-21** API Contracts in Swagger — document all endpoints with OpenAPI annotations
- **STAR-22** Exporting Figma — export Figma designs to HTML/CSS
- **STAR-23** Functional API — implement controller logic backed by EF Core and PostgreSQL
- **STAR-24** Create API Database — provision Locations DB schema and seed initial park/attraction data

Non-Goals (Potential Future Sprints)

- Mobile native app (iOS / Android)
- User photo uploads
- Waitlist and crowd forecasting